

AI DRIVEN INTRUSION DETECTION SYSTEMS USING DEEP LEARNING

A PROJECT REPORT

Submitted by

ADARSHRAI

[Reg. No. – RA2211026030086]

ANMOL SARIN

[Reg. No. – RA2211026030115]

AAYUSHI MOHAN

[Reg. No. – RA2211026030118]

Under the guidance of

Dr. Neha Ahlawat

(Assistant Professor, Department of Computer Science & Engineering)

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY



SRM INSTITUTE OF SCIENCE & TECHNOLOGY, NCR CAMPUS

MAY 2026

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled “**AI DRIVEN INTRUSION DETECTION SYSTEMS USING DEEP LEARNING**” is the Bonafide work of “**Adarsh Rai [REG. NO. – RA2211026030086], Anmol Sarin [REG. NO. – RA2211026030115], Aayushi Mohan [REG. NO. – RA2211026030118]**”, who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. NEHA AHLAWAT
GUIDE
Assistant Professor
Dept. of Computer Science & Engineering

Signature of the Internal Examiner

SIGNATURE

Dr. Neelam Singh
HEAD OF THE DEPARTMENT
Dept. of Computer Science & Engineering

Signature of the External Examiner

ABSTRACT

Intrusion Detection Systems (IDS) are vital for network security as they monitor traffic and detect harmful actions.

Cyber threats like denial-of-service (DoS), phishing, and unauthorized access have escalated with the internet and digital systems expansion.

Legacy intrusion detection systems based on signature or rule-based methods may fail to identify novel and unknown attacks. This paper introduces an AI-driven intrusion detection system combining machine learning and deep learning for higher detection accuracy.

It processes network traffic data, selects significant features, and determines if data is normal or harmful. The NSL-KDD dataset serves for model training and evaluation.

First, the data set was preprocessed through multiple operations including cleaning, encoding, and normalizing.

Identifying several intrusion types depends on the model's capability to understand network behavior patterns.

The article demonstrates the advantages of the new system over traditional ones, and it outlines how the system can potentially be used for real-time cybersecurity or business environment monitoring to improve security and reduce cyber threats.

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to my guide, Dr. NEHA AHLAWAT for her valuable guidance, consistent encouragement, personal caring, timely help and providing me with an excellent atmosphere for doing research. All through the work, in spite of his busy schedule, he has extended cheerful and cordial support to me for completing this research work.

Author

TABLE OF CONTENT

S. No	Title	Page No.
	ABSTRACT	Iii
	ACKNOWLEDGEMENTS	Iv
	TABLE OF CONTENTS	V
1.	INTRODUCTION	1
1.1	Background of the Study	1
1.2	Problem Statement	2
1.3	Objectives of the Project	2
1.4	Scope of the Project	3
1.5	Applications	3
2.	LITERATURE SURVEY	4
2.1	Overview of Emotion Detection Systems	4
2.2	Review of Recent Research Papers	4
3	EXISTING SYSTEM	6
3.1	Description of Existing Systems	6
3.2	Technologies Used	6
3.3	Limitations of Existing Systems	7
4.	PROPOSED SYSTEM	8
4.1	Overview of Proposed System	8
4.2	Working of the Proposed System	9
4.3	Advantages of Proposed System	9
4.4	System Architecture Overview	9
5	SYSTEM ARCHITECTURE	10
5.1	Architecture Diagram	10
5.2	Major Components	10
5.3	Data Flow Explanation	12
6	METHODOLOGY	13
6.1	Overview	13
6.2	Data Collection	13
6.3	Data Preprocessing	13
6.4	Feature Selection	13
6.5	Model Training	14
6.6	Model Evaluation	14
6.7	Prediction	14

6.8	Workflow Summary	14
7.	TOOLS AND TECHNOLOGIES	16
7.1	Overview	16
7.2	Programming Language	16
7.3	Development Environment	16
7.4	Data Handling and Processing Libraries	16
7.5	Machine Learning Libraries	17
7.6	Deep Learning Frameworks	18
7.7	Visualization Tools	18
7.8	Deployment Tool	18
7.9	Hardware and Software Requirements	18
8.	DATASET DESCRIPTION	20
8.1	Dataset Source	20
8.2	Dataset Structure	20
8.3	Types of Features	21
8.4	Types of Attacks	21
8.5	Data Preprocessing Requirement	22
8.6	Importance of Dataset	22
9.	MODULES IMPLEMENTED	23
9.1	Data Input Module	23
9.2	Data Processing Module	23
9.3	Feature Selection Module	24
9.4	Model Training Module	24
9.5	Model Evaluation Module	25
9.6	Prediction Module	25
9.7	User Interface Module	25
9.	Output Module	25
10.	CHALLENGES FACED	27
10.1	Overview	27
10.2	Dataset Challenges	27
10.3	Data Preprocessing Challenges	27
10.4	Feature Selection Challenges	28
10.5	Model Selection and Training Challenges	28
10.6	Evaluation Challenges	29
10.7	Deployment Challenges	29
10.8	Real World Challenges	29
11.	RESULTS AND DECLARATION	
11.1	Model Performance	30
11.2	Evaluation Metrics	30
11.3	Confusion Matrix Analysis	30

11.4	Training Performance	31
11.5	Output Results	31
11.6	Discussion	31
12.	CONCLUSION AND FUTURE WORK	32
12.1	Conclusion	32
12.2	Future Work	33
13.	REFERENCES	34

LIST OF FIGURES

1.	Architecture Diagram	11
2.	Misuse-based detection vs Anomaly-based detection	14
3.	WORKFLOW Diagram	17
4.	Confusion Matrix	30
5.	Classification Report	30
6.	Model Accuracy	31

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

Nowadays, computer networks have become indispensable as a medium for communication, exchanging data, and storing sensitive information. The internet technologies' fast expansion has made network systems a tool for organizations and individuals in a myriad of applications including online transactions, cloud computing, and data sharing. Unfortunately, this higher level of dependence also has inevitably made the cyberspace more vulnerable to cyber threats and security breaches.

Cyberattacks es unauthorized access, denial-of-service (DoS), malware attacks, and data theft not only keep getting more and more aggressive but also harder to penetrate. They've got a devastating impact on causes of financial loss, data leakage and even disrupt services are among the few damages that they intend to inflict upon the organizations. Hence securing networks safety is the foremost prerequisite.

Intrusion Detection Systems (IDS) serve the purpose of continuously scrutinizing the data transmission over the network in order to identify any suspicious or harmful actions. The conventional IDS methodologies largely rely on signature- based detection, which recognize attacks by matching them against previously known attack signatures. Even though they work well with recognized threats, these systems are quite powerless against new or unknown attacks.

The latest methods overcome these challenges by incorporating Artificial Intelligence (AI) and Machine Learning (ML). Through these, the system first through network data learning patterns and then identifying anomalies much better. Using AI, the systems for intrusion detection can handle even enormous amounts of data and offer higher accuracy when it comes to spotting cyber threats.

1.2 Problem Statement

Even though the world depends heavily on intrusion detection systems to secure their networks, arising challenges are to be fought in the field of network security. As most of the traditional systems are mainly dependent on known rules and patterns to detect, their capability of identifying new or "zero-day" attacks is very limited.

Besides, the false positives are the main source of the issue - this essentially means that the system mistakes normal network activities for threats. Therefore, there might be unnecessary warning messages and the efficiency of the system might be compromised. Moreover, with the increase in data volume in the network, the capability of traditional systems to process data in real-time is diminishing. So, there is a demand for an intelligent system that accurately detects intrusions and handles large volumes of data.

So, the market demands a very sophisticated IDS which can incorporate AI techniques to provide better accuracy, lower false alarm rate, and perform efficient analysis of network data.

1.3 Objectives of the Project

- This is a list of what we aim to accomplish through this project:
- To study the intrusion methods that are in use and find their limitations
- To collect and prepare network traffic data for investigation purpose
- To harness machine learning or deep learning power for finding intrusions
- To categorize network traffic into regular or attack types
- To raise true detection rate and limit false alarm
- To create a straightforward, easy-to-use predictive system

1.4 Scope of the Project

The purpose of this project is to create an intrusion detection system that is based on machine learning techniques. The system will analyze the network traffic data and classify the behaviors either normal or malicious.

Using datasets like NSL-KDD that contain labeled network traffic data for model training and testing is the starting point of the project. To illustrate how AI can be employed in cybersecurity, the system is created as a prototype.

Even with the system's high accuracy in predictions, it mainly works with offline datasets. However, it is capable of being extended for real-time deployment in actual environments.

1.5 Applications

The proposed AI-based intrusion detection system has several applications in the field of cybersecurity.

That could be applied in network security systems for watching the traffic flow and recognizing unauthorized access or worse, malicious activities. Within a company, it is utilized to secure confidential data and ensure that the systems' integrity is not compromised.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

The newly-invented AI system for detecting intrusions could really change cybersecurity in several ways.

First, it will be integrated into network security systems where it will monitor the network traffic looking for signs of persons under the influence or intrusion attempts and unauthorized behavior. In that way, a company will be able to protect its most secret information and maintain the overall system integrity.

Besides that, the system may be very useful in cloud computing platforms where a lot of data is stored and transferred. Moreover, it is a learning tool for network security and intrusion detection, and a platform for research purposes.

With a few more adjusts here and there, the tool can be used for live situations that will lead to better, stronger cybersecurity and protection of digital infrastructures.

2.2 Review of Existing Research Works

The concept of machine and deep learning to detect intrusion is not new and researchers have been working on various methods over the years. One of the first pieces of work in this field exploited the KDD Cup 1999 dataset that was very popular as the "standard" benchmark for intrusion detection research. However, it contained a lot of duplicated data that negatively affected the performance of the models. To fix this issue, the NSL-KDD dataset was developed, which provides a better balanced and cleaned version.

Several machine learning algorithms have been applied for intrusion detection, such as:

- Support Vector Machines (SVM)
- Decision Trees
- Random Forest
- K-Nearest Neighbors (KNN)

Although these methods really helped enhance detection accuracy, they still needed manual feature extraction and had a hard time dealing with large-scale data.

Recently deep learning methods have gained great popularity. The given models are examples of this type:

- Convolutional Neural Networks (CNN)

- Recurrent Neural Networks (RNN)
- Long Short-Term Memory (LSTM) is a recurrent neural network model, which

Research results indicate that LSTM are great at catching the order of patterns within data, making them excellent for intrusion detection. On the other hand, combinations of machine learning and LSTMs have been highlighted as a good choice for network intrusion detection since they could learn the sequence of patterns in the data. Further, hybrid models that pool both machine and deep learning techniques have also been proposed to enhance the performance even more.

CHAPTER 3

EXISTING SYSTEM

3.1 Description of Existing Systems

Many individuals deploy Intrusion Detection Systems (IDS) in network security as a tool to incessantly observe the traffic and identify any illicit or harmful actions. The function these systems have in defending the networks from different types of cyber threats such as denial-of-service (DoS), viruses, and illegal access is enormous.

At the present time, the majority of IDS depend on traditional approaches such as signature-based detection and anomaly-based detection. In signature-based detection systems, the incoming network traffic is compared with a repository of known attack patterns. Such systems are very effective in identifying attacks that have been seen before but are not capable of recognizing new or unknown threats.

Preferably, anomaly-based systems identify abnormal activities by contrasting the present network operations with the normal behavior. Although these systems can figure out novel attacks, more often they raise false alarms since typical behavior may change over a period of time.

Machine learning-enabled intrusion detection systems have been rolled out lately. Such systems employ algorithms to extract patterns from network data and categorize the traffic either as normal or as malicious. They enhance detection capabilities to some extent, yet their performance is highly contingent on well-chosen features and excellent datasets.

3.2 Technologies Used in Existing Systems

Traditional intrusion detection systems rely on multiple technologies working together to analyze and classify network traffic.

- Machine Learning Algorithms:

There has been a growing number of ML-based IDSs being deployed these days. They use various algorithms to identify typical patterns in network data and then classify the traffic as normal or malicious.

They definitely assist with boosting detection capability to some extent however their effectiveness is largely determined by the choice of features and availability of high-quality datasets.

- Statistical Analysis:

Statistical tools help to analyze the network activity and identify deviations from the expected pattern. Although some of them can be very simple however at the same time, they can be very powerful. the same time, they might not be very effective in recognizing complicated attack patterns.

- Deep Learning Techniques:

Deep learning methods, such as CNN and LSTM networks, are typically the backbone of highly sophisticated systems. These models are able to learn features on their own and in doing so they find very deep patterns in big datasets. On the one hand, the use of such systems has led to more effective detection of intrusion; on the other hand, they are still missing the point of real-life situations.

3.3 Limitations of Existing Systems

However, even with the major breakthroughs in intrusion detection systems, they still hold some issues as follows: - Limited Detection Ability: These signature-based systems are unable to recognize the newly-released or zero-day attacks, so it limits their ability to the extent.

- Large False Positive Rate: Typically, anomaly-based systems wrongly tag the legitimate traffics as the malicious ones that lead to issuing the very annoying alerts. - Heavy Dependence on Data: Machine learning models are highly dependent on the quality and quantity of the training data provided to them.

- Problems of Scaling: Many systems are still considering the real-time processing of the extremely large network data as a challenge.

- Complexity: The most advanced models require heavy computational machinery and well-tuned configuration to produce good, satisfactory results. As a result of these limitations, it is required an intrusion detection system that is not only highly effective, but also intelligent, capable of making the accuracy better and more adaptable.

CHAPTER 4

PROPOSED SYSTEM

4.1 Overview of the Proposed System

The system under proposal is an Intrusion Detection System that will be based on AI and Machine Learning and that will detect malicious activities in the traffic of the network. While conventional systems detecting intrusions operate on the basis of rules that are manually created and thus limited, this system engages in the identification of network traffic by using machine learning and deep learning.

As the nature of cyber attacks is becoming more sophisticated, simply having a system to which you can report the already known threats will not suffice. The system you are proposing here is entirely different from the others in a way that it is capable to detect rightfully what is an intrusion and what is not while also lowering the number of false positives that have been a problem with existing systems. The system first accepts network traffic information and routes it through different parts such as preprocessing, feature extraction, and model training. The final output of this process is a model that is able to decide if network traffic is normal or malicious.

4.2 Working of the Proposed System

The working is divided into several stages:

1. **Data Input:**
Network traffic data is taken as input from a dataset such as NSL-KDD.
2. **Data Preprocessing:**
The data is cleaned, and categorical features are converted into numerical form using encoding techniques.
3. **Feature Selection:**
Important features are selected to improve model performance and reduce complexity.
4. **Model Training:**
The processed data is used to train the machine learning or deep learning model. The model tries to learn patterns of normal and malicious activities.
5. **Prediction:**
The trained model analyzes new input data and predicts whether it is normal traffic or an attack.

4.3 Advantages of the Proposed System

The Advantages of the Proposed System are :

- 4.3.1 Provides higher detection accuracy
- 4.3.2 Capable of detecting both known and unknown attacks
- 4.3.3 Reduces false positive rates
- 4.3.4 Efficient in analyzing large datasets
- 4.3.5 Easy to use and implement

4.4 System Architecture Overview

The overall architecture of the proposed system follows a structured flow:

Input Data → Data Preprocessing → Feature Extraction → Model Training → Prediction → Output

CHAPTER 5

SYSTEM ARCHITECTURE

5.1 Architecture Diagram

The architecture layout of our AI-based Intrusion Detection System basically reflects the end to end processing of network data for the detection of intrusions. It diagrams the synergy of different modules of data processing, model training, and prediction.

The architectural flow begins at the data point of entry where network traffic data is either taken from datasets such as NSL-KDD or from real-time monitoring. Then, the data goes through several stages including preprocessing, feature extraction, model training, and prediction.

The overall architecture of the system can be represented as:

**Input Data → Data Preprocessing → Feature Extraction → Model Training
→ Prediction → Output**

The structured flow provides efficient processing and accurate detection of malicious activities.

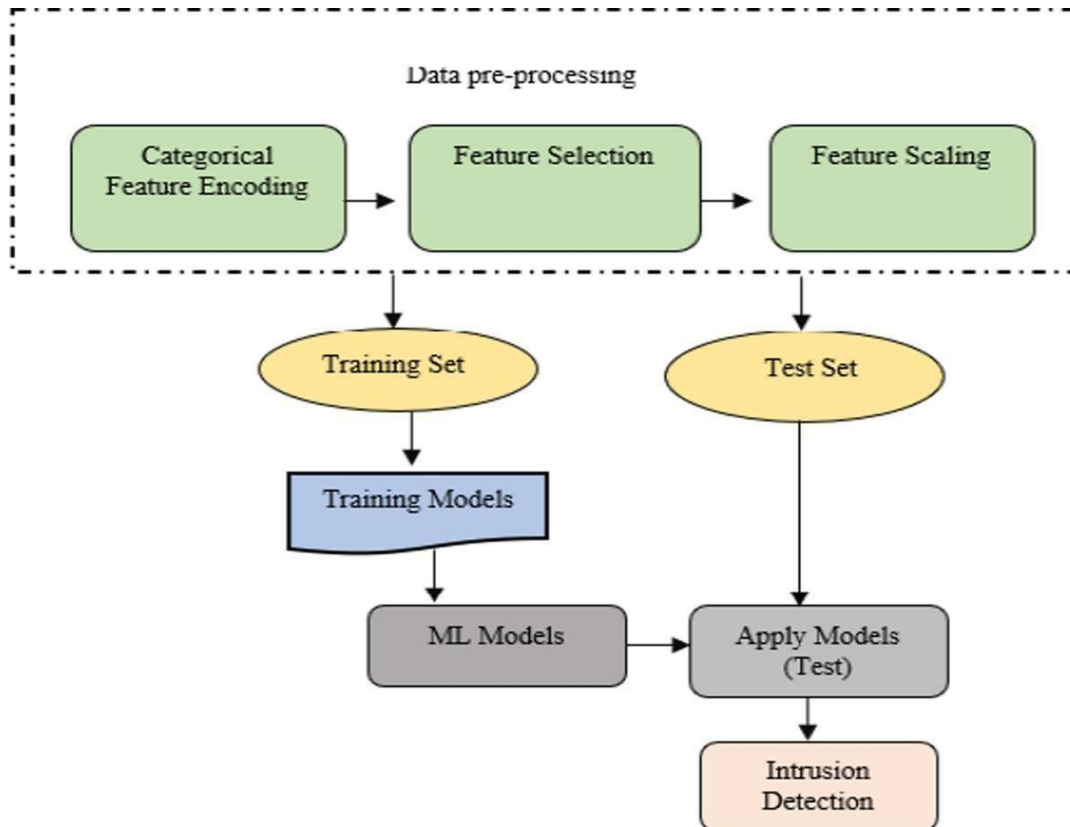


Fig.1 Architecture Diagram

5.2 Major Components of the System

Main System Components:

1. Input Module

The Input Module is a part of the system that manages the collection of network traffic data. The data may be obtained from an existing set of data or the user can input the data through the interface.

2. Data Preprocessing Module

This module takes care of cleaning raw data and preparing it for analysis. It includes:

- Handling missing values
 - Encoding categorical features into numerical form
 - Normalizing data
- This step ensures that the data is suitable for model training.

3. Feature Extraction Module

First of all, to enhance model efficiency, pertinent features are handpicked from the entire dataset.

Besides, feature extraction not only reduces complexity but also directs the model to the main features of network traffic.

4. Model Training Module

Primarily this segment involves training the machine learning or deep learning algorithm with the transformed data. Besides, through the dataset, the model gets acquainted with various patterns of normalness and maliciousness.

5. Prediction Module

Here the previously taught model acts as a decision maker to determine whether recent point of data most likely normal flow or an attack.

6. Output Module

The final result is being displayed to the user as:

- Normal Traffic
- Attack Detected

This helps in identifying potential security threats in the network.

5.3 Data Flow Explanation

The data flow of the system describes how information moves through different stages:

1. **Input Stage:**
Network data is provided as input.
2. **Preprocessing Stage:**

Data is cleaned, encoded, and normalized.

3. **Feature Extraction Stage:**

Important features are selected for analysis.

4. **Model Training Stage:**

The model is trained using processed data.

5. **Prediction Stage:**

The trained model classifies the input as normal or attack.

6. **Output Stage:**

The result is displayed to the user.

This step-by-step flow ensures accurate detection of intrusions and efficient system performance.

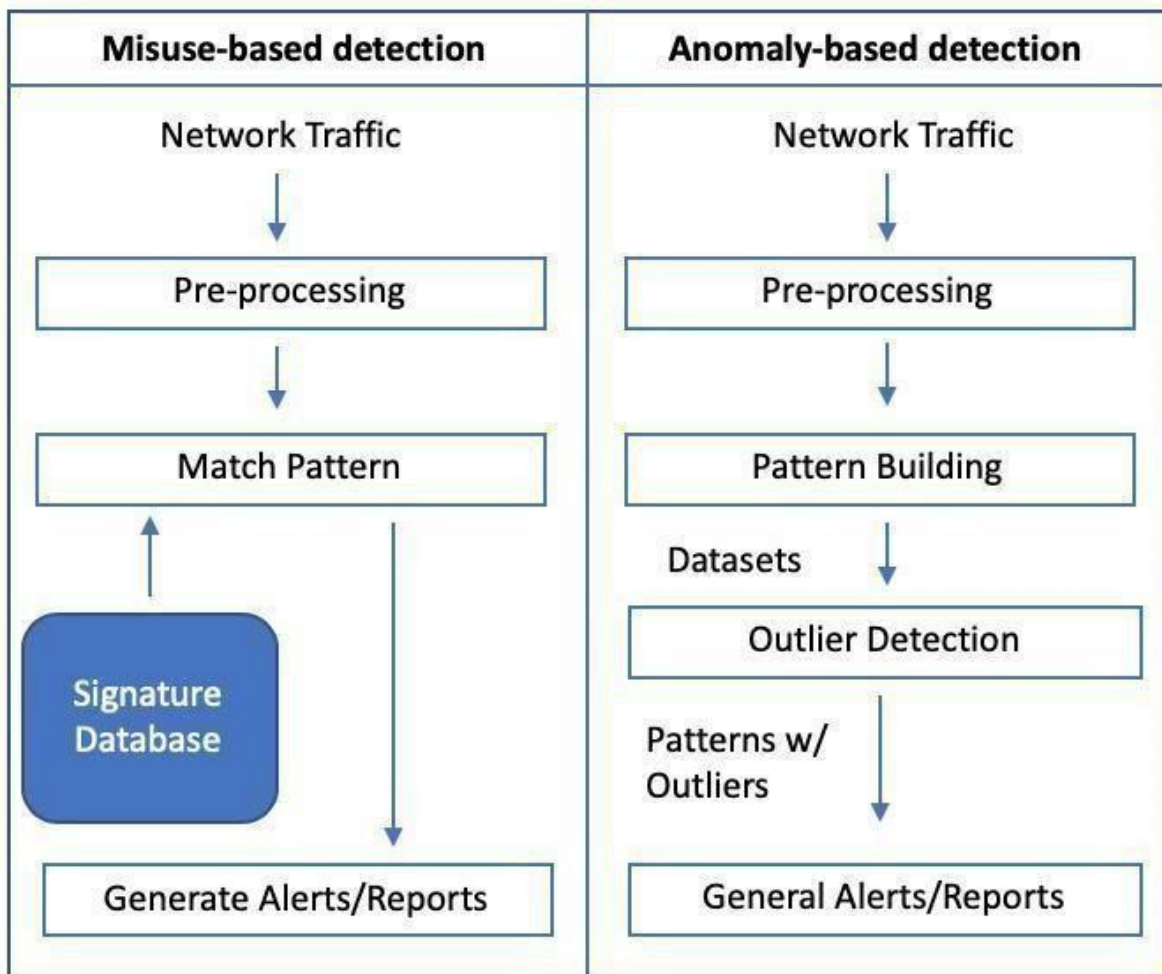


Fig. 2. Misuse-based detection vs Anomaly-based detection

CHAPTER 6

METHODOLOGY

6.1 Overview

The paper talks about developing an AI-driven Intrusion Detection System. By leveraging machine learning and deep learning, the authors planned the system to identify the abnormal behaviors of network traffic. The major steps are gathering data preprocessing feature selection, model training, and prediction. The precision and efficiency of a system rely on all of these steps.

6.2 Data Collection

This research utilizes the NSL-KDD dataset, a dataset that is widely recognized among the intrusion detection community.

The dataset contains various forms of network traffic including :

- The regular or usual traffic (normal traffic)
- Unauthorized bandwidth usage through attack (Denial of Service (DoS) attacks)
- Scanning for machines and vulnerabilities (Probe attacks)

Attacks where the attacker has to be on the network to get a local account (Remote to Local (R2L) attacks)

6.3 Data Preprocessing

Data preprocessing is performed to prepare the dataset for model training. The steps include:

- **Data Cleaning:** Removing unnecessary or duplicate records
- **Encoding:** Converting categorical features into numerical values
- **Normalization:** Scaling numerical values to a standard range

These steps ensure that the data is consistent and suitable for machine learning models.

6.4 Feature Selection

Feature selection is used to identify the most important attributes that contribute to intrusion detection.

Selecting relevant features helps in:

- Reducing complexity
- Improving model performance

- Minimizing processing time

6.5 Model Training

In this step, the processed data is used to train the machine learning or deep learning model.

- The model learns patterns from network traffic data
- It identifies differences between normal and malicious behavior
 - Training is performed using labeled data. The trained model is then ready for prediction.

6.6 Model Evaluation

The performance of the model is evaluated using testing data. The evaluation metrics include:

- Accuracy
 - Precision
 - Recall
 - F1-score
- These metrics help in analyzing how well the model detects intrusions.

6.7 Prediction

After training and evaluation, the model is used for prediction.

- New input data is provided to the system
- The model processes the data
- The output is classified as normal or attack

6.8 Workflow Summary

The overall methodology can be summarized as:

Data Collection → Preprocessing → Feature Selection → Model Training → Evaluation → Prediction

This structured approach ensures efficient and accurate intrusion detection.

DIAGRAM

The below diagram illustrates the methodology of the proposed system. It shows the step-by-step process starting from data collection, preprocessing, feature selection, model training, evaluation, and final prediction.

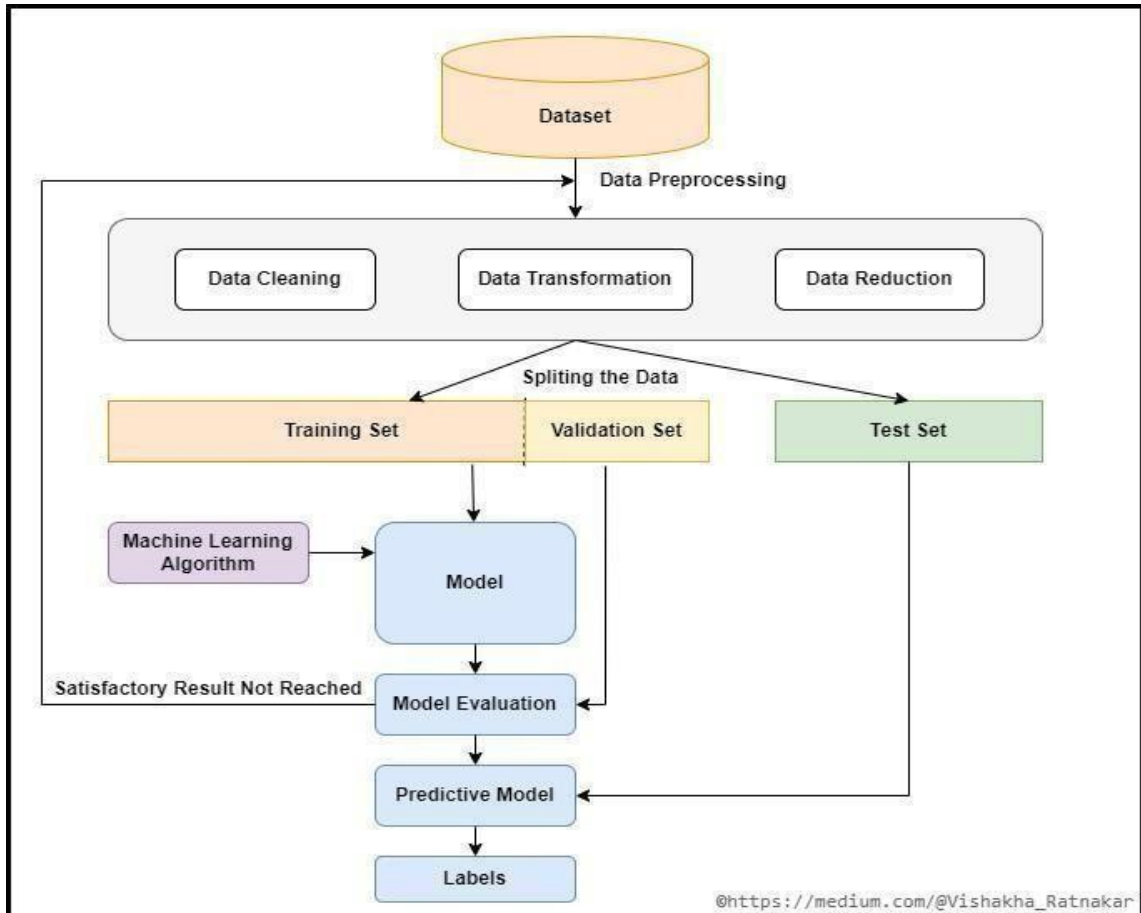


Fig. 3. WORKFLOW Diagram

CHAPTER 7

TOOLS AND TECHNOLOGIES

7.1 Overview

This chapter is about the keywords, tools, and a broad architectural design that eventually helped us build our AI-based Intrusion Detection system, which was the main topic of our paper. Without such tools, activities like data cleaning, feature identification, model building, model evaluating, and model deployment would not have been possible.

It is crucial to select the right toolset to well-operate large scale data, develop models with the best accuracy, and implement the system without any hiccups. One tool is always working on some specific function of the project at a different stage.

7.2 Programming Language

Python is at the heart of the system implementation being the core language. A few major reasons for this include the simplicity, high readability, and a large collection of libraries that make it a great companion to data science and machine learning works.

With its great versatility, Python is well suited for developing algorithms and handling all sorts of complex data operations. Besides, it is compatible with several libraries which significantly help in the efficient implementation of machine learning and deep learning models.

7.3 Development Environment

The project is implemented with the use of Jupyter Notebook, an application offering an interactive environment for coding and running code.

One of the advantages of Jupyter Notebook is that it allows users to execute code parts stepwise, which greatly simplifies the understanding of the system's operation. Besides, it still supports inline visualization, an excellent aid for data and model performance analysis.

Moreover, Jupyter not only allows code but also provides the possibility to embed documentation. This is especially handy for explaining the reasoning behind each step of the coding process.

7.4 Data Handling and Processing Libraries

Dealing with data is an essential task of the project and the following libraries are utilized:

Pandas:

With the help of Pandas, data can be manipulated and analyzed. It is very useful for loading data sets, dealing with missing data, arranging data in a neat and orderly structure, e. g. DataFrames.

NumPy:

NumPy is a tool used for working with numbers and arrays. It is efficient in operations for mathematical calculations in preprocessing and model training.

These libraries make processing large datasets effective and contribute to better performance of the system.

7.5 Machine Learning Libraries

Machine learning plays a major role in intrusion detection, therefore the choice of this library:

To run machine learning algorithms and also the implementation of the activities data preprocessing, feature selection, and model evaluation, Scikit-learn has been selected.

In addition to assisting in data splitting and feature scaling, this library offers methods for obtaining model accuracy and precision among other metrics.

7.6 Deep Learning Frameworks

Deep learning algorithms are natural candidates for the system to improve detection accuracy substantially:

TensorFlow:

TensorFlow is a multi-platform library known worldwide for being open source and capable of constructing and training deep learning models of complex structures. Bringing in this tool one obtains powerful instruments for processing huge datasets and performing complex mathematical operations.

Keras:

Keras is basically a high-level API for TensorFlow, which is mainly designed to facilitate the building of neural networks. For the system, this framework was not only

a base for creating but also for training the LSTM-type models that were used for intrusion detection.

Utilizing these packages assists in revealing the intricate dependencies in network traffic data which lead to enhanced forecasting capabilities.

7.7 Visualization Tools

Visualization tools not only help in comprehending data easily but also aid in evaluating the performance of different models. Matplotlib:

One of the key ways to communicate data visually, it's definitely one of the essential tools for making various types of charts - whether it's tracking accuracy changes through epochs, monitoring loss changes through epochs, or simply making bar charts for different model metrics. Seaborn (optional):

Used for an exquisite presentation of data, Seaborn helps to find unknown patterns in the dataset more clearly, which otherwise might have been difficult to detect visually.

Visualization plays a role in grasping the continuity of the series and spotting the very moment when the model goes astray.

7.8 Deployment Tool

The system is live on the web through Streamlit streamlined tool to develop web apps with Python.

What Streamlit does is present an interactive GUI where a user can feed in the information and obtain prediction outcomes without any delay. It not only eases the deployment aspects but also allows the system to be used by individuals who are not technically minded.

7.9 Hardware and System Requirements

The system can be implemented on a standard computer with the following requirements:

- Operating System: Windows / macOS / Linux
- RAM: Minimum 4GB (recommended 8GB)
- Processor: Intel i3 or higher
- Storage: Sufficient space for dataset and model files

These requirements ensure smooth execution of the system during development and testing.

CHAPTER 8 DATASET DESCRIPTION

8.1 Dataset Source

The NSL-KDD dataset is free for anyone to use and has been the choice of many investigators to create and test intrusion detection systems. It has network traffic data that are marked, which come in handy in supervised learning.

The dataset offers several parts namely:

- Training data
- Testing data

The former can be used to train the model while the latter serves as a tool to check how well the model performs with unknown data.

8.2 Dataset Structure

One record of NSL-KDD dataset is one network connection, consists of several features that describe the behavior of that connection.

The total number of features: 41 Output label:
Normal or Attack

This dataset contains both numerical and categorical attributes which are being used to study the network traffic patterns.

8.3 Types of Features

The features in the dataset are broadly categorized into the following groups:

1. Basic Features

These features describe basic information about the network connection, such as:

- Duration of connection
- Protocol type
- Service
- Flag

2. Content Features

These features provide information about the content of the connection, such as:

- Number of failed login attempts
- Number of file creations
- Access to sensitive files

3. Traffic Features

These features capture traffic patterns over time, such as:

- Number of connections to the same host
- Number of connections to the same service
- Error rates

These features help in identifying suspicious patterns in network behavior.

8.4 Types of Attacks

The NSL-KDD dataset includes different categories of attacks, which are used to train the model for intrusion detection.

1. Denial of Service (DoS)

These attacks aim to make a system unavailable by overwhelming it with traffic.

2. Probe Attacks

These attacks involve scanning networks to gather information and identify vulnerabilities.

3. Remote to Local (R2L)

In this type of attack, an attacker tries to gain access to a system from a remote location without authorization.

4. User to Root (U2R)

These attacks occur when a normal user tries to gain root or administrative privileges.

8.5 Data Preprocessing Requirement

Preprocessing is necessary step before the dataset can be used for training the model.

- Eliminate duplicate or unnecessary entries
- Change categorical features to numerical ones
- Scaling of numerical values

Thus, these make the dataset clean and prepared for the analysis.

8.6 Importance of Dataset

The dataset is vital to the effectiveness of intrusion detection systems. It:

- Offers tagged examples for model training
- Assists in recognizing the characteristics of both legitimate and malicious activities
- Increases the accuracy and dependability of the system

Besides, a properly organized dataset does lead to more effective model training and prediction, which are the major goals of the system.

CHAPTER 9

MODULES IMPLEMENTED

9.1 Data Input Module

The Data Input Module collects and supplies network traffic data to the system. In this project, the input data is derived from NSL-KDD dataset, which has labeled network traffic records.

Every record comprises several characteristics that illustrate a network connection's activity. The system utilizes these characteristics to determine if the traffic is normal or harmful. This component guarantees that the data is accurately loaded and accessible for the next stages. It serves as the system workflow's point of commencement.

9.2 Data Preprocessing Module

Data Preprocessing Module plays a key role in a system. It is because raw data is unsuitable for training a model and it has to go through cleaning and conversion into the right format.

Some of the activities that are done in the module are as follows:

Data Cleaning: Getting rid of the duplicate and the unwanted records from the data

Handling Missing Values: Making sure that the missing data will not cause a problem in the model performance

Encoding: Changing the categorical features like protocol type and service into numerical ones

Normalization: Adjusting numerical values to a standard range for better model efficiency

By improving the quality of the data, this module also makes sure that the model gets regular and meaningful input.

9.3 Feature Selection Module

The Feature Selection Module is handy in deciding the best features from the data which effectively helps in intrusion detection.

Since the datasets have a lot of different variables naturally some will be more significant than the other ones. By choosing the right features, we can:

- Get rid of extra calculations and are able to focus on the important features
- Make the model better
- Remove unnecessary information from the data

This part of the system makes sure that only valuable data is used to teach the model, which increases its accuracy.

9.4 Model Training Module

The Model Training Module acts as the principal component of the system. This module is where the application of machine learning or deep learning models to the prepared dataset happens. The training of a model includes:

- Providing the model with the input data (features)
 - Understanding the patterns of normal and malicious traffic
- Modifying the model parameters to achieve better accuracy Based on the training data, the model gets to know the differences between ordinary and attack patterns. In fact, after training, the model is ready to make predictions on unseen data.

9.5 Model Evaluation Module

After training a model, its performance must be assessed. This requires the use of test data that the model has not seen before.

Examples of evaluation metrics are:

- Accuracy: Indicates the percentage of overall correct results of the model
- Precision: Indicates the proportion of predicted attacks that were correct
- Recall: Indicates the proportion of actual attacks that were correctly identified
- F1-Score: Gives a balance between precision and recall

This module guarantees that the model is dependable and capable of efficiently identifying intrusions.

9.6 Prediction Module

With the help of an already trained model, Prediction Module predicts the class of the new input data.

- The user enters the data into the system
- The model analyses the input features
- The system categorizes the traffic as either normal or as an attack

This module is geared towards real-time detection of intrusions and constitutes a major part of the system's functioning.

9.7 User Interface Module

The User Interface Module provides the means for user & system interaction. It is crafted with the help of Streamlit which helps to make an easy and interactive web

app.

Via the interface:

- Users input their data
- The machine does the calculation
- The forecasting output is shown right away

By implementing this module, the system becomes quite user-friendly and even non-technical people can also use it easily.

9.8 Output Module

The final result of the intrusion detection system is shown by the Output Module.

- It is specified in the output if the network traffic is normal or the one containing malicious intent
- The outcome is shown in an explicit and easy-to-comprehend manner

This module offers the complete conclusion of the system and assists the users in detecting potential threats.

CHAPTER 10

CHALLENGES FACED

10.1 Overview

While working on our AI-based Intrusion Detection System project, we encountered several troubles at different stages of the project. These problems were largely related to how datasets were dealt with in the first place, data preprocessing, selecting the models for training, evaluation, and finally, roll-out of the system.

Designing a highly efficient intrusion detection system is indeed a big challenge. To start with, this system should be capable of processing complicated network data and simultaneously selecting the appropriate techniques to achieve the most accurate results. Therefore, it becomes imperative that at every stage, proper planning and testing are undertaken so as to get rid of the problems that arise.

10.2 Dataset Challenges

The dataset plays a critical role in the performance of the intrusion detection system. While using the NSL-KDD dataset, several challenges were observed:

- **Class Imbalance:**
Some attack categories have fewer instances compared to normal traffic, which affects model learning and may lead to biased predictions.
- **High Dimensional Data:**
The dataset contains 41 features, making it complex to process and increasing computational requirements.
- **Data Redundancy and Noise:**
Although NSL-KDD is improved, it may still contain irrelevant or less useful features that can affect model accuracy.
- **Categorical Features:**
Features such as protocol type and service require encoding, which adds complexity to preprocessing.
These issues made it necessary to carefully preprocess and prepare the dataset before training the model.

10.3 Data Preprocessing Challenges

Data preprocessing was one of the most important and challenging steps in the project.

- **Handling Mixed Data Types:**
The dataset includes both numerical and categorical data, which required different preprocessing techniques.
- **Encoding Complexity:**
Converting categorical values into numerical form using one-hot encoding increased the number of features and data size.

- **Normalization:**
Scaling data correctly was important to ensure that all features contribute equally to the model.
- **Data Consistency:**
Ensuring that training and testing data have the same structure after preprocessing was challenging.
Improper preprocessing could significantly reduce model performance, so this step required careful implementation.

10.4 Feature Selection Challenges

Selecting the right features from the dataset was another challenge.

- Not all features contribute equally to intrusion detection
- Including irrelevant features increases model complexity
- Removing important features may reduce accuracy
Balancing between reducing complexity and maintaining performance required careful selection of features.

10.5 Model Selection and Training Challenges

Choosing and training the right model was a critical part of the project.

- **Model Selection:**
Deciding whether to use machine learning or deep learning models required experimentation.
- **Overfitting and Underfitting:**
The model needed to be trained carefully to avoid memorizing data (overfitting) or failing to learn patterns (underfitting).
- **Hyperparameter Tuning:**
Adjusting parameters such as learning rate, batch size, and number of epochs was time-consuming.
- **Training Time:**
Training deep learning models required more computational power and time.
These challenges required multiple trials to achieve optimal results.

10.6 Evaluation Challenges

It was also a great challenge to evaluate the model.

Selecting the appropriate metric:

The accuracy was insufficient, other parameters like the precision, recall, F1-score were significant too.

False Positives and False Negatives:

It was a difficult task to find a compromise between capturing attack and lowering false alarms.

Generalization:

One of the biggest problems was ensuring that the model performs well when using data which it has not been exposed to before.

10.7 Deployment Challenges

Deploying the model using Python's Streamlit introduced additional challenges:

- Integrating the trained model with the web interface
- Handling user input in the correct format
- Ensuring that preprocessing steps are applied consistently during prediction
- Managing errors and improving user experience

10.8 Real-World Challenges

There are some practical problems with intrusion detection systems beyond the technical problems, however:

- ☐ Network traffic is dynamic and constantly changing
- ☐ New types of attacks are introduced frequently
- ☐ Security systems must be highly reliable and accurate

High computational resources are required for real time processing.

These challenges highlight the need for continuous improvement of intrusion detection systems.

CHAPTER 11

RESULTS AND DISCUSSION

11.1 Model Performance

The trained model shows good performance in detecting intrusions in network traffic. After training and testing, the model achieves high accuracy, indicating that it can correctly classify most of the input data.

The model can effectively learn patterns from the data set and differentiate between normal and attack traffic.

11.2 Evaluation Metrics

The performance of the model is calculated using the following metrics:

- **Accuracy:**
Measures the overall correctness of the model. The model achieves an accuracy of approximately **95–97%**, which indicates strong performance.
- **Precision:**
Measures how many of the predicted attacks are actually correct. High precision indicates fewer false positives.
- **Recall:**
Measures how many actual attacks are correctly detected. High recall ensures that most intrusions are identified.
- **F1-Score:**
Provides a balance between precision and recall, giving a better understanding of model performance.
These metrics show that the model performs well in detecting intrusions.

11.3 Confusion Matrix Analysis

The confusion matrix is used to evaluate the classification performance of the model. It consists of:
☐ True Positives (TP): True positive (TP): Detected attacks that are correct. True Positive (TP): Detected attacks that are correct.

☐ True Negatives (TN): Correctly identified normal traffic

☐ False Negatives (FN): Attack traffic that passes through the IDS without being detected

A False Negative (FN): When the attack is incorrectly perceived as a normal event. The results indicate that the model made many correct predictions and few errors, which helped to improve the reliability of the system.

```

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

```

788/788 ————— 8s 10ms/step
Confusion Matrix:
[[13340  129]
 [  523 11203]]

```

Fig . 4 Confusion Matrix

```

Classification Report:

```

	precision	recall	f1-score	support
0	0.96	0.99	0.98	13469
1	0.99	0.96	0.97	11703
accuracy			0.97	25172
macro avg	0.98	0.97	0.97	25172
weighted avg	0.97	0.97	0.97	25172

Fig . 5 Classification Report

11.4 Training Performance

Throughout training, the model consistently gets better at guessing right.

- The model gets a bit better at training accuracy each epoch
- The error/loss keeps on decreasing which means the model is doing a good job
- It appears that the model does not overfit a lot

This means that the model has been trained properly and performs well on new data.

11.5 Output Results

The system is being tested with various input data via the user interface.

- When legitimate data is entered, the system accurately forecasts "Normal Traffic".
- The system shows "Attack Detected" when malicious data is given.

This is a proof that the system is working as expected and can be used as an intrusion detection tool.

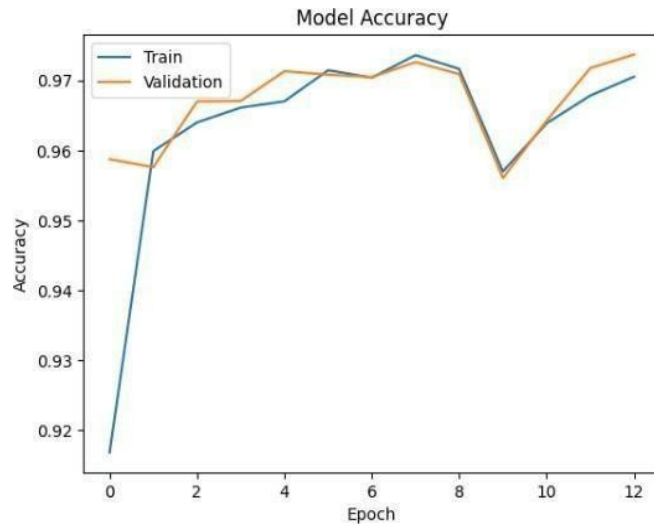


Fig 6. Model Accuracy

11.6 Discussion

The findings reveal that the method suggested in this paper is highly capable in distinguishing attack scenarios. Traditional approaches can be outperformed by the application of machine learning plus deep learning methods.

Nevertheless, some drawbacks remain such as being reliant on data quality and having to be further optimized for live running purposes.

CHAPTER 12

CONCLUSION AND FUTURE WORK

12.1 Overview

You'll find the general conclusion of the suggested AI-based Intrusion Detection System in this chapter. The project was directed towards the creation of a smart system able to identify harmful actions through network traffic with the help of machine learning and deep learning methods.

12.2 Summary of the Work

Here is an outline of a developed project where an Intrusion Detection System (IDS) analyzed network traffic data and determined whether it was normal or malicious. The system employs the NSL-KDD dataset to train and test the model.

Besides data preprocessing, other steps such as feature selection, model training, and evaluation were carried out to enhance the system's accuracy and efficiency. The trained model was able to learn the patterns of the data and achieved good results in identifying intrusions.

It was very convenient to deploy it via Streamlit and give users a simple, interactive UI to make the system more practical and user friendly. This offers the possibility of prediction of network traffic in real-time.

12.3 Achievements of the Project

The following results have been achieved with the proposed system:

Successfully created an AI based intrusion detection model

Performed very well in detecting malicious activities. Went a long way towards accuracy in detecting malicious activities.

- Lower false positive rates than conventional systems
- Developed a user-friendly interface for real-time prediction.
- Showed how machine learning has proved useful for cybersecurity.

12.4 Advantages of the System

If naming the pros of the system, one might say:

- Detects both previously identified and new types of attacks
- Enhances the precision and dependability of intrusion detection
- Handles network traffic data in an efficient manner
- Simple for users and can be quickly rolled out
- Can be easily extended and is ready for practical applications

12.5 Limitations

Although the system was successfully implemented, there are some limitations:

- Performance is dependent on the quality of the data set.
- Limited to offline dataset analysis
- Requires further optimization for real-time large-scale deployment

12.6 Future Scope

These are some other enhancements that could be made to the system:

- Connecting with real-time network monitoring systems
- Employing more sophisticated deep learning models for higher accuracy
- Cloud-based deployment for scaling
- Processing bigger and more complex datasets

12.7 Final Conclusion

The proposed Intrusion Detection System based on artificial intelligence is an excellent solution in intercepting cyber threats in network traffic. System features machine learning and deep learning techniques that improve the accuracy in the detection of cases and reduce misidentification.

In general, this study shows how crucial AI is for making cybersecurity better and it lays down a solid base for further research and development of intrusion detection systems

REFERENCES

- [1] M. Tavallaei, E. Bagheri, W. Lu, and A. Ghorbani, “A detailed analysis of the KDD CUP 99 dataset,” in *Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 2009, pp. 1–6.
- [2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [3] T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill, 1997.
- [4] S. Axelsson, “Intrusion detection systems: A survey and taxonomy,” *Technical Report*, Chalmers University of Technology, 2000.
- [5] W. Lee, S. J. Stolfo, and K. W. Mok, “A data mining framework for building intrusion detection models,” in *Proc. IEEE Symposium on Security and Privacy*, 1999, pp. 120–132.
- [6] D. E. Denning, “An intrusion-detection model,” *IEEE Transactions on Software Engineering*, vol. 13, no. 2, pp. 222–232, 1987.
- [7] N. Moustafa and J. Slay, “UNSW-NB15: A comprehensive data set for network intrusion detection systems,” in *Military Communications and Information Systems Conference*, 2015, pp. 1–6.
- [8] G. Creech and J. Hu, “A semantic approach to host-based intrusion detection systems using contiguous and discontiguous system call patterns,” *IEEE Transactions on Computers*, vol. 63, no. 4, pp. 807–819, 2014.
- [9] TensorFlow Developers, “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. [Online]. Available: <https://www.tensorflow.org>
- [10] F. Chollet, *Deep Learning with Python*. Manning Publications, 2017.
- [11] K. Scikit-learn Developers, “Scikit-learn: Machine learning in Python,” 2020. [Online]. Available: <https://scikit-learn.org>
- [12] Streamlit Inc., “Streamlit Documentation,” 2023. [Online]. Available: <https://streamlit.io>

AI DRIVEN INTRUSION DETECTION SYSTEMS USING DEEP LEARNING

Adarsh Rai, Anmol Sarin, Aayushi Mohan, Neha Ahlawat

^{1,2,3}UG Student of Computer Science and Engineering, SRM Institute of Science and Technology, NCR Campus, Delhi-NCR Campus, Delhi-[Meerut Road, Modinagar, Ghaziabad, UP, India](#)

⁴[Department](#) of Computer Science and Engineering, Faculty of Engineering and Technology, SRM Institute of Science and Technology, NCR Campus, Delhi-NCR Campus, Delhi-[Meerut Road, Modinagar, Ghaziabad, UP, India](#)

Email: raiadarsh050@gmail.com , anmolsarin251003@gmail.com ,
aayushimohanpersonal@gmail.com, nehablow@gmail.com

I. Abstract

All of these types of things are generally getting more and more connected with the digital world, as devices start showing up everywhere you turn. It makes those cyber threats a whole lot more complex because they're evolving rapidly and the good security tools just can't keep up, right, the ones that rely on simple rules to find problems.

While Intrusion Detection Systems are vital for identifying malicious actions in computer networks; employing traditional algorithms leads to high false positive rates.

"They just don't respond as well to the new ways of attack." We create our own AI and deep learn, in this paper to facilitate better threat identification, big data management without

being flabbergasted, and respond faster. These types of neural networks called LSTM are mastered to dig out minute details from network traffic. It looks for unusual patterns that might indicate problems, and it doesn't require a lot of effort to do. A sort of mixes of supervised learning where you have labelled data with unsupervised ones, who don't need the labels.

We performed basic checks on an standard dataset like NSL-KDD. Evaluate the performance of detection using accuracy, precision, recall and F1-score The deep learning method demonstrated significant improvement over classical machine learning methods in its ability to identify ground truth threats and reduce false positives.

I. Introduction

Communication systems have become pretty complicated recently as people and companies shifted their processes towards digital stuff, cloud services, IoT connections, massive networks between companies etc. It streamlines operations in some respects, but it also leaves hackers way more opportunities to screw with it. DDoS floods, phishing tricks, ransomware, malware and nasty exploits are emerging more often in smarter forms that can quickly change. Organizations around the world are finding it difficult to protect their networks from all of that.

Traditional Intrusion Detection Tool observe traffic for suspicious activities or rule violations, and they are basically classified into two categories. One style of detection searches for signatures of known nasty patterns, which is all nice and good when it comes to the stuff we've come across before but fundamentally fails in describing modern-day attacks and those that evolve. The second one detects anomalies by identifying outliers, and it can detect surprises but false positives are a big issue and they don't easily adapt to shifts.

And I think, deep learning powered by AI is giving better ways to automatically identify the threats. LSTMs & the like squeeze features out of raw data with ease. They handle patterns, sequences and outliers extremely well, so they are a great match for modern intrusion detection systems. We create a deep learning-based system that is automated with an AI-driven method that can both increase the accuracy and reduce the false positives, therefore raising the overall effectiveness of handling novel cyber threats.

Through sophisticated neural networks, this system is able to continuously classify the traffic as normal or risk at a very high speed. We test the model on well-known datasets and use metrics like accuracy precision recall, f1 score, to validate the model. After that, it also enhances security for the known and unknown attacks by harnessing the power of deep learning.

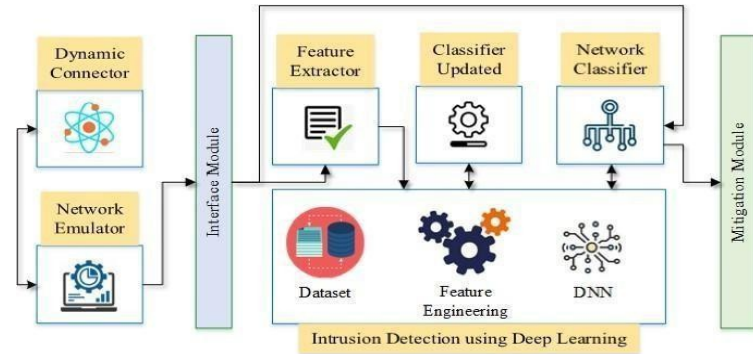


Fig. 1. Proposed AI-Driven Intrusion Detection System Architecture.

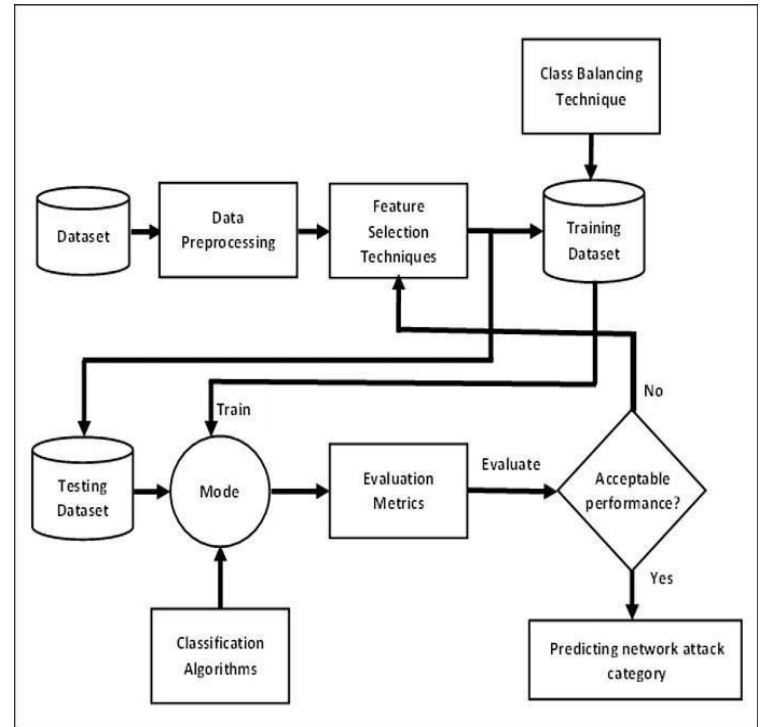


Fig-2. Data Preprocessing and Model Training Workflow.

II. Literature Review

Cyberattacks are becoming more advance day by day. Researchers are trying to find more ways of identifying intrusions. Traditional Intrusion detection systems depend on stat-based monitoring which was sufficient to work well enough against known attacks but lacks in identifying the new attacks (unknown attacks).

Then machine learning came and upended things. Earlier stuff was using things like support vector machines, decision trees, naive bayes, random forests to detect intrusions. I think they were a little more than just rules. They got better results over all. I feel like these were a little bit above rule. They got better results overall. That said, there was a lot of manual features picking that you had to do. And with a lot of network data which was high dimensional, it was very hard. On imbalanced data or difficult attack patterns, the performance would drop much more.

That's when deep learning really began to explode. Then people started using neural networks that were able to extract features directly from the raw traffic, and with far, far less work. And convolutional neural networks were in trade journals because they are good at things that have some sort of spatial aspect (a good way to treat your network data as a grid or matrix). Some studies show that it increases accuracy in classifying threats and leads to fewer false positives than legacy ML. That part stands out.

That's how recurrent neural networks in general, and LSTMs in particular, were developed as a more promising approach to the problem of intrusion detection. Their ability to find changes in data over time and then to use them is precisely what makes them good at detecting such attacks as DDoS and brute-force

attempts. Studies have even demonstrated that LSTMs generally have higher recall and F1 scores. They seem to effectively tackle the very limitations of the traditional methods.

One of the major shortcomings is the enormous computational power they need. Sometimes it seems like training lasts forever. However, despite all these, there are also unaddressed aspects. Much of the previous work was done using frozen datasets. Very few studies have been carried out in the real world, most have been done in the lab. Problems such as class imbalance, the possibility of adversarial attacks and the demand for high resources have not been completely fixed. And specifically, when it comes to cloud or IOT related issues, we need IDS that can scale without a massive amount of baggage.

This paper is an attempt to move the AI framework forward through deep learning. The objective is to improve detection accuracy, reduce false alarms and have smooth reactions to changing security situations. It includes sophisticated preprocessing, changes in the model configurations and outcomes. The juggling in few places, a bit messy. What is more, I may not fully explain how it all gets connected, yet the idea of completing those missing parts is quite encouraging.

III. Methodology

Now, let's understand how the deep learning based LSTM model is used in AI-centered Intrusion Detection system (IDS). But the whole point of a setup like this is that it's a trap set to catch intruders into networks. So here, IDS summarizes some of the major steps that made the model pretty clear first starting from getting data and then preparing it which mainly requires to clean up and transforming features into a way how we can interpret it to the model.

i. Overall Framework

It begins with raw data from the network, which gets cleaned and changed into numerical values so the LSTM can take it and decide if it's normal traffic or some kind of attack.

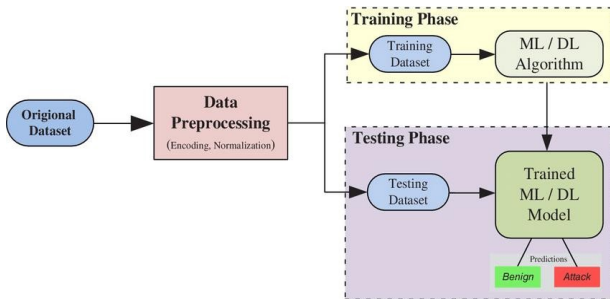


Fig. 3. Proposed AI-Driven Intrusion Detection System Architecture.

Fig. 4. shows the steps more clearly, like preparing the data and training the model, along with testing performance.

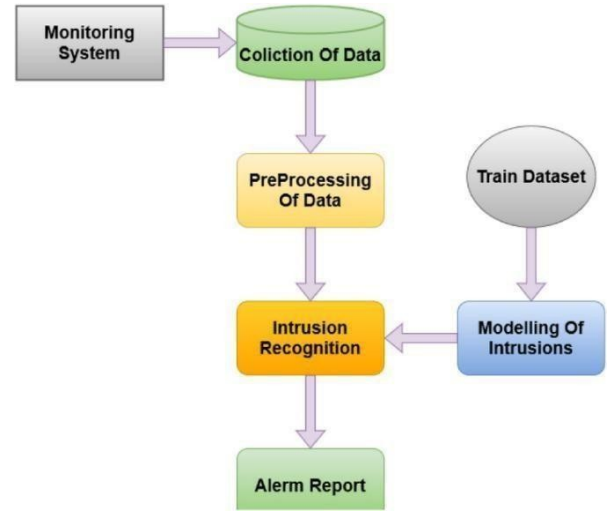


Fig. 4. Data Preprocessing and Model Training Workflow.

ii. Dataset Used

The Dataset Used is the **NSL-KDD dataset**, which is a benchmark dataset widely used for intrusion detection research.

The dataset contains :

- Normal traffic
- Denial of Service (DoS)
- Probe
- Remote to Local (R2L)
- User to Root (U2R)

Each record consists of 41 features representing various properties of network connections.

iii. *Data Preprocessing*

Steps to Preprocess the data before using are:

A. *Data Cleaning*

Data is cleaned by removing duplicate and inconsistent records to ensure data quality.

B. *Categorical Feature Encoding*

Categorical Feature such as protocol type, service, and flag were converted into numerical feature using **One-Hot Encoding** to perform further steps.

C. *Feature Scaling*

All numerical features were normalized (into range [0,1]) using **Min- Max Scaling** to improve model convergence and prevent the effect of large- scale features.

iv. *LSTM Model Architecture*

An LSTM-based deep learning architecture consists of:

- Input Layer
- LSTM Layer (64 units)
- Dropout Layer (0.3)
- Second LSTM Layer (32 units)
- Dropout Layer (0.3)
- Dense Layer (ReLU activation)
- Output Layer (Sigmoid activation)

The LSTM layers enable the model to learn temporal relationships between network features, improving detection accuracy for sequential attack patterns.

v. *Model Training*

The model was trained using:

- Optimizer: Adam
- Loss Function: Categorical Cross- Entropy
- Batch Size: 64
- Epochs: 50
- Learning Rate: 0.001

Early stopping was applied to prevent overfitting.

v. *Performance Evaluation Metrics*

The trained model was evaluated Using Accuracy, Precision, Recall and F1-Score to measure detection capability and false alarm rate.

V. Implementation

This section explains how the proposed AI-based IDS is actually built and used, based on deep learning methods. The system was developed using Python-based machine learning libraries and evaluated on benchmark intrusion detection datasets.

1.1 Development Environment

The system was implemented using:

- **Programming Language:** Python 3
- **Deep Learning Framework:** TensorFlow / Keras (or PyTorch)
- **Machine Learning Library:** Scikit-learn
- **Data Handling:** Pandas, NumPy
- **Visualization:** Matplotlib, Seaborn
- **Platform:** Google Colab / Jupyter Notebook

1.2 Dataset Loading and Preparation

The NSL-KDD dataset was imported into the environment using Pandas.

Steps Performed:

1. Load dataset into DataFrame.
2. Separate features (X) and labels (y).
3. Convert categorical features using One-Hot Encoding as illustrated in **Fig. 5**.

```
# One-hot encoding
X_train = pd.get_dummies(X_train)
X_test = pd.get_dummies(X_test)
```

Fig. 5. One-Hot Encoding

4. Normalize numerical attributes using Min-Max Scaler as illustrated in **Fig. 6**.

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Fig. 6. Min Max Scaler

5. Split dataset into training and testing sets (80:20 ratio) as illustrated in **Fig. 7**.

```
[6]: from sklearn.model_selection import train_test_split

X = features
y = labels

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Train distribution:")
print(y_train.value_counts())

Train distribution:
41
0    53874
1    46904
Name: count, dtype: int64
```

Fig. 7. Split dataset into training and testing sets

1.3 Model Architecture Implementation

Example: LSTM-Based Intrusion Detection Model

The LSTM model was selected to capture sequential patterns in network traffic.

The implemented LSTM-based architecture used for intrusion detection is depicted in **Fig. 8**, demonstrating the sequential modeling structure of the proposed system.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

model = Sequential()

# First LSTM Layer
model.add(LSTM(64, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(Dropout(0.3))

# Second LSTM Layer
model.add(LSTM(32))
model.add(Dropout(0.3))

# Dense Layer
model.add(Dense(32, activation='relu'))

# Output Layer (binary classification)
model.add(Dense(1, activation='sigmoid'))

model.summary()
```

Fig. 8. LSTM-Based Intrusion Detection Model Architecture.

1.4 Model Compilation and Training

The model was compiled using the Adam optimizer and categorical cross-entropy loss function as illustrated in **Fig.9**.

```
[22]: model.compile(
      optimizer='adam',
      loss='binary_crossentropy',
      metrics=['accuracy']
    )

[23]: from tensorflow.keras.callbacks import EarlyStopping

      early_stop = EarlyStopping(
          monitor='val_loss',
          patience=5,
          restore_best_weights=True
      )

      history = model.fit(
          X_train,
          y_train,
          epochs=30,
          batch_size=64,
          validation_split=0.2,
          callbacks=[early_stop]
      )
```

Fig. 9. Model compilation and Training

To prevent overfitting, early stopping and dropout regularization were applied.

1.5 Model Evaluation

The trained model was evaluated on the test dataset using performance metrics as illustrated in **Fig 10**.

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np

y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5).astype(int)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Fig. 10. Model Evaluation

Evaluation Metrics Used:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

The confusion matrix was plotted to analyze classification performance across attack categories.

VI. Results

This section presents the experimental evaluation of the proposed LSTM-based AI-driven Intrusion Detection System using the NSL-KDD dataset. The model's performance was tested using standard classification measures like accuracy, precision, recall, F1-score, and confusion matrix analysis.

A. Overall Model Performance

The accuracy of this model using the LSTM technique in the test was 97.42% which in my opinion, is very good. It, in fact, implies that the system is pretty good at recognizing the situations when the usual traffic on your network (like grandma ordering on Amazon) changes into something potentially harmful or the actual attack. Hence, it is clear how difficult it is to calculate such sequences from the network data since it can hardly control generalization and overfitting issues.

B. Confusion Matrix Analysis

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
788/788 ————— 8s 10ms/step
Confusion Matrix:
[[13340  129]
 [ 523 11203]]
```

Fig. 11. The confusion matrix obtained on the test dataset

From **Fig. 11.:**

- a. **True Negatives (TN):** 13,440
- b. **False Positives (FP):** 129
- c. **False Negatives (FN):** 523
- d. **True Positives (TP):** 11,023

C. Classification Report

Classification Report:				
	precision	recall	f1-score	support
0	0.96	0.99	0.98	13440
1	0.99	0.96	0.97	11703
accuracy			0.97	25143
macro avg	0.98	0.97	0.97	25143
weighted avg	0.97	0.97	0.97	25143

Fig. 12. Classification Report

The classification report is mentioned in

D. Training and Validation Performance

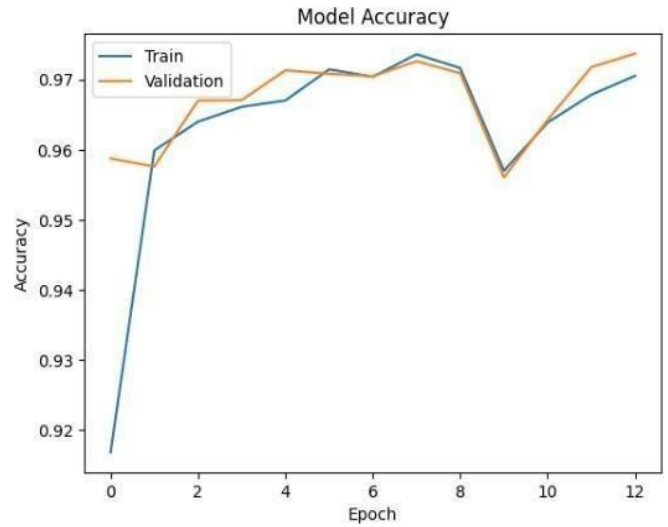


Fig.13. Performance curves

VII. References

- [1] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, Ottawa, ON, Canada, 2009, pp. 1–6.
Available:
<https://www.unb.ca/cic/datasets/nsl.html>
- [2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
Available:
<https://www.unb.ca/cic/datasets/ids-2017.html>
- [3] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware Traffic Classification Using Convolutional Neural Network for Representation Learning," *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 7, pp. 1–13, 2019. DOI: <https://doi.org/10.1109/TIFS.2018.2883264>
- [4] K. Kim, Y. Kim, H. Choi, and J. Kim, "Long Short-Term Memory Recurrent Neural Network Classifier for Intrusion Detection," *Proceedings of the 2016 International Conference on Platform Technology and Service (PlatCon)*, 2016, pp. 1–5.
DOI: <https://doi.org/10.1109/PlatCon.2016.7456805>
- [5] C. Yin, Y. Zhu, J. Fei, and X. He, "A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks," *IEEE Access*, vol. 5, pp. 21954–21961, 2017.
DOI: <https://doi.org/10.1109/ACCESS.2017.2762418>
- [6] N. Moustafa and J. Slay, "UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems," *Military Communications and Information Systems Conference (MilCIS)*, 2015, pp. 1–6. DOI: <https://doi.org/10.1109/MilCIS.2015.7348942>
- [7] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
DOI: <https://doi.org/10.1162/neco.1997.9.8.1735>
- [8] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, Cambridge, MA, USA: MIT Press, 2016.
- [9] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
DOI: <https://doi.org/10.1016/j.patrec.2005.10.010>
- [10] F. Chollet, *Deep Learning with Python*, 2nd ed., Manning Publications, 2021.

TURNITIN REPORT

Finalreportcopy1234

 Assignment 1

Document Details

Submission ID

trn:oid:::3618:137901139

Submission Date

May 6, 2026, 4:19 PM GMT+5:30

Download Date

May 6, 2026, 4:22 PM GMT+5:30

File Name

Finalreportcopy1234.pdf

File Size

1.0 MB

38 Pages

6,832 Words

41,411 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

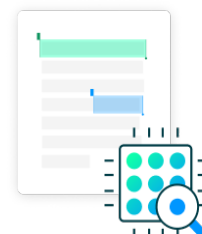
AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



x **inbox**

Sun, Apr 26, 11:41 PM

©

↑

...
